

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE			DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week1 – Monday	Batch	23CSBTB47B
Name	Manoj Aelishala	Hall Ticket No	2303A54054
Assignment Number:1.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI Lab Objectives: ❖ To explore and evaluate the functionality of Google Gemini for AI-assisted coding within Google Colab. ❖ To understand and use Cursor AI for code generation, explanation, and refactoring. ❖ To compare outputs and usability between Gemini, GitHub Copilot, and Cursor AI. ❖ To perform code optimization and documentation using AI tools. Lab Outcomes (LOs): After completing this lab, students will be able to: ❖ Generate Python code using Google Gemini in Google Colab. ❖ Analyze the effectiveness of code explanations and suggestions by Gemini. ❖ Set up and use Cursor AI for AI-powered coding assistance. ❖ Evaluate and refactor code using Cursor AI features. ❖ Compare AI tool behavior and code quality across different platforms. Task 1: Statistical Summary for Survey Data ❖ Scenario: You are a data analyst intern working with survey responses stored as numerical lists. ❖ Task: Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values. ➤ Prompt : You are an AI coding assistant. Write a Python function in Google Colab that: Accepts a list of numbers as input Calculates the mean, minimum, and maximum values	Week1 - Monday	

Prints the results clearly

Then run the function on the list [12, 7, 9, 21, 5] and show the output.

❖ **Expected Output:**

Input List: [12, 7, 9, 21, 5]

Mean: 10.80

Minimum: 5

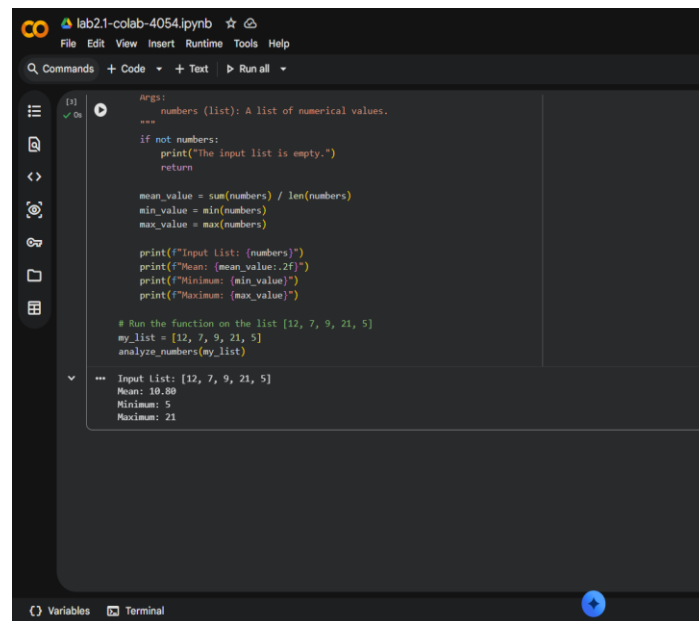
Maximum: 21

➤ Output shown in Colab

```
# Run the function on the list [12, 7, 9, 21, 5]
my_list = [12, 7, 9, 21, 5]
analyze_numbers(my_list)

Input List: [12, 7, 9, 21, 5]
Mean: 10.80
Minimum: 5
Maximum: 21
```

➤ Screenshot of Gemini prompt and result



```
def analyze_numbers(numbers):
    """
    numbers (list): A list of numerical values.
    """
    if not numbers:
        print("The input list is empty.")
        return

    mean_value = sum(numbers) / len(numbers)
    min_value = min(numbers)
    max_value = max(numbers)

    print(f"Input List: {numbers}")
    print(f"Mean: {mean_value:.2f}")
    print(f"Minimum: {min_value}")
    print(f"Maximum: {max_value}")

# Run the function on the list [12, 7, 9, 21, 5]
my_list = [12, 7, 9, 21, 5]
analyze_numbers(my_list)
```

Input List: [12, 7, 9, 21, 5]
Mean: 10.80
Minimum: 5
Maximum: 21

Task 2: Armstrong Number – AI Comparison

❖ **Scenario:**

You are evaluating AI tools for numeric validation logic.

❖ **Task:**

Generate an **Armstrong number checker** using **Gemini** and **GitHub Copilot**. Compare their outputs, logic style, and clarity.

- ❖ **Expected Output:**
 - 153 IS an Armstrong number.
 - 123 IS not an Armstrong number.

Side-by-side comparison table

Point	Google Colab Version	Code (Return-based)
Function ending	print("153 is an Armstrong number")	Same both
Function usage	Can't reuse result in other logic	Can reuse in if, loops, tests
Main logic location	Logic + output mixed together	Logic inside function, output outside

➤ Screenshots of prompts and generated code

lab2.1-4054.py

lab2.1-4054.py

is_armstrong_number

43

Task 2

44

'''

45

Prompt: Armstrong Number Checker

46

Write a Python function that checks if a number is an Armstrong number.

47

Test it with 153 and 123, and print the results clearly.

48

'''

49

50

def is_armstrong_number(number):

51

'''

52

Check if a number is an Armstrong number.

53

Ctrl+L to chat, Ctrl+K to generate

54

An Armstrong number is a number that is equal to the sum of its own digits

55

each raised to the power of the number of digits.

56

57

Args:

58

number (int): The number to check

59

60

Returns:

61

bool: True if the number is an Armstrong number, False otherwise

62

'''

63

Convert number to string to get individual digits

64

num_str = str(number)

65

num_digits = len(num_str)

66

67

Calculate sum of each digit raised to the power of num_digits

68

armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)

69

70

Check if the sum equals the original number

71

return armstrong_sum == number

72

73

Test the function with given numbers

74

test_numbers = [153, 123]

75

76

for num in test_numbers:

77

result = is_armstrong_number(num)

78

status = "is" if result else "is not"

79

print(f"{num} {status} an Armstrong numb

80

81

Review next file

Python list statistic: Python list

Task: Armstrong Number Checker

Write a Python function that checks if a number is an Armstrong number.

Test it with 153 and 123, and print the results clearly.

Ran command: python

\$ python lab2.1-4054.py

Statistics for the given numbers:

Mean: 10.80

Minimum: 5

Maximum: 21

153 is an Armstrong number

123 is not an Armstrong number

✓ Success

Thought for 2s

Perfect! I've successfully implemented the Armstrong number checker. Here's what the function does:

Function Logic:

- Converts the number to a string to easily access each digit
- Counts the number of digits (n)
- Calculates the sum of each digit raised to the power of n
- Returns True if the sum equals the original number, False otherwise

Test Results:

- 153 is an Armstrong number because $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$
- 123 is not an Armstrong number because

> 1 File

Review

Plan, @ for context, / for commands

Grok Code

Former lab

Ln 53, Col 1

Spaces: 4

UTF-8

CRLF

() Python

3.12.10 64-bit (Microsoft Store)

Go Live

```
def is_armstrong_number(number):  
    """  
    Checks if a given number is an Armstrong number and prints the result.  
  
    An Armstrong number (also known as a narcissistic number) is a number  
    that is the sum of its own digits each raised to the power of the  
    number of digits.  
  
    Args:  
        number (int): The number to check.  
    """  
    num_str = str(number)  
    n = len(num_str)  
    sum_of_powers = 0  
    for digit in num_str:  
        sum_of_powers += int(digit) ** n  
  
    if sum_of_powers == number:  
        print(f"{number} IS an Armstrong number.")  
    else:  
        print(f"{number} IS not an Armstrong number.")  
  
    # Run the function on the numbers 153 and 123  
    is_armstrong_number(153)  
    is_armstrong_number(123)  
  
    """  
    153 IS an Armstrong number.  
    123 IS not an Armstrong number.
```

Task 3: Leap Year Validation Using Cursor AI

❖ Scenario:

You are validating a calendar module for a backend system.

❖ Task:

Use **Cursor AI** to generate a Python program that checks whether a given year is a leap year. Use **at least two different prompts** and observe changes in code.

Expected Detailed Explanations:

1900:

- Is divisible by 4 ✓
- Is a century year (divisible by 100) ✓
- Is NOT divisible by 400 ✗
- **Result: NOT a leap year**

2000:

- Is divisible by 4 ✓
- Is a century year (divisible by 100) ✓
- Is divisible by 400 ✓
- **Result: IS a leap year**

2024:

- Is divisible by 4 ✓
- Is NOT a century year ✗
- **Result: IS a leap year**

2100:

- Is divisible by 4 ✓
- Is a century year (divisible by 100) ✓
- Is NOT divisible by 400 ✗
- **Result: NOT a leap year**

Two versions of code

lab2.1-4054.py

lab2.1-4054.py > check_leap_year

```
83 # Task 3
84 """
85 Prompt 1 :
86 Write a Python function is_leap_year(year) that returns True if the year is a leap year, otherwise False.
87 Test it with 2000, 2021, 2024, and 2100, and print the results.
88 """
89
90 def is_leap_year(year):
91     """
92     Check if a year is a leap year.
93
94     A year is a leap year if:
95     - It is divisible by 4, AND
96     - It is NOT divisible by 100, OR it IS divisible by 400
97
98     Args:
99         year (int): The year to check
100
101     Returns:
102         bool: True if the year is a leap year, False otherwise
103     """
104     # A year is a leap year if it is divisible by 4
105     # but not by 100 unless it is also divisible by 400
106     if year % 4 == 0:
107         if year % 100 == 0:
108             return year % 400 == 0
109         else:
110             return True
111     else:
112         return False
113
114 # Test the function with given years
115 test_years = [2000, 2021, 2024, 2100]
116
117 for year in test_years:
118     result = is_leap_year(year)
119     status = "is" if result else "is not"
120     print(f"{year} {status} a leap year")
121
```

Review next file >

lab2.1-4054.py

lab2.1-4054.py > check_leap_year

```
131 """
141 Returns:
142     None: Prints detailed explanation of leap year status
143 """
144 print(f"\nChecking year: {year}")
145 # Step 1: Check if divisible by 4
146 if year % 4 != 0:
147     print(f" {year} is not divisible by 4.")
148     print(f" Therefore, {year} is NOT a leap year.")
149     return
150
151 print(f" {year} is divisible by 4.")
152
153 # Step 2: If divisible by 4, check if it's a century year (divisible by 100)
154 if year % 100 == 0:
155     print(f" {year} is a century year (divisible by 100).")
156
157     # Step 3: For century years, must be divisible by 400
158     if year % 400 == 0:
159         print(f" {year} is also divisible by 400.")
160         print(f" Therefore, {year} IS a leap year.")
161     else:
162         print(f" {year} is not divisible by 400.")
163         print(f" Therefore, {year} is NOT a leap year.")
164 else:
165     print(f" {year} is not a century year.")
166     print(f" Therefore, {year} IS a leap year.")
167
168 # Test the function with given years
169 test_years = [1900, 2000, 2024, 2100]
170
171 print("Leap Year Analysis:")
172 print("=" * 50)
173
174 for year in test_years:
175     check_leap_year(year)
176
```

Review next file >

Python list statistic: Python list + ⌚ ...

Create a Python program check_leap_year(year) with a docstring and comments explaining the leap year

1900:

- Is divisible by 4 ✓
- Is a century year (divisible by 100) ✓
- Is NOT divisible by 400 X
- Result: NOT a leap year

2000:

- Is divisible by 4 ✓
- Is a century year (divisible by 100) ✓
- Is divisible by 400 ✓
- Result: IS a leap year

2024:

- Is divisible by 4 ✓
- Is NOT a century year X
- Result: IS a leap year

2100:

- Is divisible by 4 ✓
- Is a century year (divisible by 100) ✓
- Is NOT divisible by 400 X
- Result: NOT a leap year

When you run the code, it will provide clear, educational explanations showing the step-by-step reasoning for each year's leap year status, making it perfect for understanding

> 1 File Review

Plan, @ for context, / for commands

🔍 Grok Code

Cursor-fab Ln 144, Col 38 Spaces: 4 UTF-8 CRLF Python 3.12.10 64-bit (Microsoft Store) Go Live

➤ Brief comparison :

Key Difference in Code Style

```
if year % 4 == 0:
    if year % 100 == 0:
        return year % 400 == 0
    else:
        return True
else:
    return False
```

Compact style

```
return (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
```

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

❖ Scenario:

Company policy requires developers to write logic before using AI.

❖ Task:

Write a Python program that calculates the **sum of odd and even numbers in a tuple**, then refactor it using any AI tool.

Prompt:

Write a Python program to find the sum of odd numbers and sum of even numbers in a tuple using basic logic.

Then refactor the code using an AI tool to make it simpler.

Show:

1. Original code

```
# Original code (student logic)
def calculate_sums_original(numbers):
    """
    Original implementation using basic loop logic
    """
    even_sum = 0
    odd_sum = 0

    for num in numbers:
        if num % 2 == 0:
            even_sum += num
        else:
            odd_sum += num

    return even_sum, odd_sum
```

2. Refactored code

```
# Refactored code (AI version) - More efficient and cleaner
def calculate_sums_refactored(numbers):
    """
    Refactored implementation using list comprehensions and built-in sum function
    """
    even_sum = sum(num for num in numbers if num % 2 == 0)
    odd_sum = sum(num for num in numbers if num % 2 != 0)
    return even_sum, odd_sum

# Test both versions
test_tuple = (1, 2, 3, 4, 5, 6)
```

3. Short explanation of changes

Input: (1, 2, 3, 4, 5, 6)

❖ Expected Output:

```
PS C:\Users\alish\OneDrive\Documents\Desktop\Winter training> & C:/Users/alish/
.exe "c:/Users/alish/OneDrive/Documents/Desktop/Winter training/lab2.1-4054.py"
=== ORIGINAL CODE (Student Logic) ===
Sum of even numbers: 12
Sum of odd numbers: 9

=== REFACTORED CODE (AI Version) ===
Sum of even numbers: 12
Sum of odd numbers: 9
○ PS C:\Users\alish\OneDrive\Documents\Desktop\Winter training>
```

➤ Explanation of improvements

1. Code is **shorter and cleaner**
2. Removes manual looping
3. Uses built-in sum() for better readability